APACHE AIRFLOW

YAML DAGs | DAG Factory | Astronomer Astro | CI/CD Deploy

Skill Level	Duration	Airflow Version	Platform
Intermediate	3-4 Hours	Airflow 2.9+	Astronomer Astro

Enterprise AI Training Program

Data Engineering and MLOps Track

TABLE OF CONTENTS

Lab Modules and Topics

01 | LAB OVERVIEW AND PREREQUISITES

Understanding DAG Factory and Astronomer Astro

What is DAG Factory?

DAG Factory is an open-source library (maintained by Astronomer) that lets you define Apache Airflow DAGs declaratively using YAML or JSON configuration files instead of writing Python code. This separates pipeline structure from business logic, making DAGs easier to manage, review, and scale across large teams.

Advantage	Description
No Python boilerplate	Define DAGs in YAML with no imports, no with-blocks, no scaffolding
Generate at scale	One config file can spin up dozens of DAGs with different parameters
Non-engineer friendly	Data analysts can create pipelines without deep Python knowledge
Clean Git diffs	YAML configs produce minimal, readable diffs on every change
Dynamic pipelines	Jinja templating and Airflow Variables drive data-driven workflows

Architecture: How DAG Factory Works

DAG Factory reads YAML config files when Airflow starts. The DagFactory class parses each config and generates DAG objects that Airflow registers in its DagBag — exactly as if you had written them in Python. One Python loader file handles everything.

Layer	Component	Role
Config	dags/config/*.yml	Declarative pipeline definitions in YAML
Library	dag-factory (PyPI)	Parses YAML and generates DAG Python objects
Entry Point	dags/dag_factory_loader.py	Single .py file that loads all configs
Airflow	DagBag + Scheduler	Discovers and schedules all generated DAGs
Deployment	Astronomer Astro	Runs DAGs in production cloud environment

Prerequisites

* Astronomer Astro CLI installed (astro version confirms it)
* Active Astro deployment running (Cloud or local via astro dev start)
* Python 3.10+ with pip available in your shell
* Git installed for version-controlled config management
* Basic Airflow familiarity: DAGs, Tasks, Operators, and schedules

INFO

Verify your Astro setup: astro dev ps (should show 3 containers: webserver, scheduler, postgres)
Astro local UI: http://localhost:8080 | Default credentials: admin / admin

02 | PROJECT SETUP AND INSTALLATION

Creating your Astro project and configuring DAG Factory

1 Verify Astro CLI and active deployment

Terminal — Verify Astro Environment

```
# Confirm Astro CLI is installed
astro version

# List your active Astronomer deployments
astro deployment list

# Sample output:
# NAME              ID                  STATUS
# my-airflow-prod   clxxxxxxxxxxxxx     HEALTHY

# Verify local dev environment is running
astro dev ps
# Expected: webserver, scheduler, postgres all in Running state
```

2 Create a new Astro project

Terminal — Initialize Project

```
# Create and enter project directory
mkdir airflow-dagfactory-lab && cd airflow-dagfactory-lab

# Initialize Astro project scaffold
astro dev init

# Generated structure:
# airflow-dagfactory-lab/
# +-- dags/              <- DAG Python/YAML files go here
# +-- plugins/           <- Custom operators, hooks, listeners
# +-- include/           <- Shared config, SQL, scripts
# +-- requirements.txt   <- Python package dependencies
# +-- packages.txt       <- OS-level packages (apt)
# +-- Dockerfile         <- Astro Runtime base image
# +-- .astro/            <- Astro project metadata
```

3 Install dag-factory via requirements.txt

requirements.txt (project root)

requirements.txt

```
# requirements.txt
dag-factory>=0.19.0

# Provider packages for operators used in YAML DAGs
apache-airflow-providers-http>=4.6.0
apache-airflow-providers-postgres>=5.6.0
apache-airflow-providers-amazon>=8.0.0
pandas>=2.0.0
```

TIP

dag-factory 0.19+ supports Airflow 2.7 and above with full TaskFlow API compatibility.
Pin the version (>=0.19.0) for reproducible builds across dev and production environments.

4 Create config directory and loader file

Terminal — Create Structure

```
# Create the YAML config directory
mkdir -p dags/config

# Final project structure after setup:
# dags/
# +-- dag_factory_loader.py    <- universal loader (one file)
# +-- config/
#     +-- etl_pipeline.yml     <- Module A DAG
#     +-- multi_pipeline.yml   <- Module B DAGs (multiple)
#     +-- sensor_pipeline.yml  <- Module B advanced DAG
```

5 Create the universal DAG Factory loader

dags/dag_factory_loader.py

dags/dag_factory_loader.py

```
# dags/dag_factory_loader.py
# Single file — auto-loads ALL YAML configs from dags/config/

import dagfactory
from pathlib import Path
import glob

# Resolve config directory relative to this file
CONFIG_DIR = Path(__file__).parent / 'config'

# Auto-discover all .yml files in config directory
yaml_files = glob.glob(str(CONFIG_DIR / '*.yml'))

# Generate DAGs into Airflow's global namespace
for yaml_file in yaml_files:
    factory = dagfactory.DagFactory(yaml_file)
    factory.clean_dags(globals())
    factory.generate_dags(globals())

print(f'DAG Factory loaded {len(yaml_files)} config file(s)')
```

SUCCESS

This loader pattern is the Astronomer-recommended approach: one .py file, unlimited YAML DAGs.

Every new .yml added to dags/config/ is automatically discovered on scheduler restart.

03 | MODULE A: CONVERT PYTHON DAG TO YAML DAG

Hands-on: Replace boilerplate Python with clean YAML

Use Case: E-Commerce Order ETL Pipeline

We build a realistic ETL pipeline that extracts orders from an API, validates the data, transforms it with Pandas, loads it to a database, and sends a notification. First we examine the Python version, then convert it fully to YAML.

Step A1 — The Original Python DAG (for reference only)

`dags/python_etl_dag.py — reference, do NOT deploy this version`

Original Python DAG (reference)

```python
# ORIGINAL PYTHON DAG — 60+ lines of boilerplate
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from airflow.operators.empty import EmptyOperator

default_args = {
    'owner': 'data-team',
    'depends_on_past': False,
    'email_on_failure': True,
    'email': ['alerts@company.com'],
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    dag_id='order_etl_pipeline',
    default_args=default_args,
    schedule_interval='0 6 * * *',
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=['etl', 'orders', 'production'],
) as dag:

    extract = BashOperator(
        task_id='extract_orders',
        bash_command='python /opt/scripts/extract.py {{ ds }}',
    )
    validate = PythonOperator(
        task_id='validate_data',
        python_callable=validate_orders,
    )
    transform = PythonOperator(
        task_id='transform_data',
        python_callable=transform_orders,
    )
    load = BashOperator(
        task_id='load_to_db',
        bash_command='python /opt/scripts/load.py',
    )
    notify = EmptyOperator(task_id='send_notification')

    extract >> validate >> transform >> load >> notify
```

Step A2 — The YAML Equivalent via DAG Factory

dags/config/etl_pipeline.yml

dags/config/etl_pipeline.yml

```yaml
# dags/config/etl_pipeline.yml
# Exact equivalent of the Python DAG above — much cleaner

order_etl_pipeline:
  description: 'E-Commerce Order ETL: extract, validate, transform, load'
  schedule_interval: '0 6 * * *'
  start_date: '2024-01-01'
  catchup: false
```

```
  tags:
    - etl
    - orders
    - production
  default_args:
    owner: data-team
    depends_on_past: false
    email_on_failure: true
    email:
      - alerts@company.com
    retries: 2
    retry_delay_sec: 300

  tasks:
    extract_orders:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /opt/scripts/extract.py {{ ds }}'
      dependencies: []

    validate_data:
      operator: airflow.operators.python.PythonOperator
      python_callable_file: /opt/scripts/validate.py
      python_callable_name: validate_orders
      dependencies: [extract_orders]

    transform_data:
      operator: airflow.operators.python.PythonOperator
      python_callable_file: /opt/scripts/transform.py
      python_callable_name: transform_orders
      dependencies: [validate_data]

    load_to_db:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /opt/scripts/load.py'
      dependencies: [transform_data]

    send_notification:
      operator: airflow.operators.empty.EmptyOperator
      dependencies: [load_to_db]
```

Python DAG vs YAML DAG: Side-by-Side

Aspect	Python DAG	YAML DAG (DAG Factory)
Lines of code	~60 lines	~40 lines (33% reduction)
Import statements	6-10 imports required	Zero imports needed
Adding a new task	Write Python + wire dependencies	Add YAML block + dependencies list
Non-engineer usable	Requires Python knowledge	Readable by any team member
Git diff on change	Complex Python diffs	Clean, minimal YAML diffs
Boilerplate	DAG context manager, default_args dict	Clean key-value structure

Step A3 — Validate and Test Locally

Terminal — Test Locally

```
# Start local Astro environment
astro dev start

# Open Airflow UI: http://localhost:8080  |  admin / admin
# Verify 'order_etl_pipeline' appears in the DAGs list

# Trigger a test run from CLI
astro dev run dags trigger order_etl_pipeline

# Check run history
astro dev run dags list-runs --dag-id order_etl_pipeline

# Validate YAML config is parseable
astro dev run python -c "
import dagfactory
f = dagfactory.DagFactory('dags/config/etl_pipeline.yml')
f.generate_dags(globals())
print('Config is valid!')"
```

NOTE

If DAG does not appear: check for YAML indentation errors (use 2 spaces, never tabs).

Run `astro dev logs scheduler` to see DAG import errors in real time.

DagBag refreshes every ~30 seconds. Wait before assuming the DAG is missing.

04 | MODULE B: GENERATE MULTIPLE DAGs FROM YAML

Hands-on: One config file, many independent pipelines

Use Case: Regional Sales Data Pipelines

A very common real-world pattern is running the same pipeline logic for multiple business units, regions, or data sources. With DAG Factory you define multiple top-level keys in a single YAML file and Airflow registers each as an independent, separately-schedulable DAG.

Step B1 — Multi-DAG YAML (3 regional pipelines, 1 file)

dags/config/multi_pipeline.yml

dags/config/multi_pipeline.yml (Part 1 of 2)

```yaml
# dags/config/multi_pipeline.yml
# Generates 3 independent DAGs from this single config file

# ■■ Shared defaults using YAML anchor ■■■■■■■■■■■■■■■■■■
default: &default_config
  schedule_interval: '0 8 * * 1-5'
  start_date: '2024-01-01'
  catchup: false
  default_args:
    owner: analytics-team
    retries: 3
    retry_delay_sec: 120
    email_on_failure: true
    email: [data-alerts@company.com]

# ■■ DAG 1: North Region ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sales_pipeline_north:
  <<: *default_config
  description: 'Daily sales ETL for North Region'
  tags: [sales, north, daily]
  tasks:
    extract_north_sales:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/extract.py --region north --date {{ ds }}'
      dependencies: []
    transform_north:
      operator: airflow.operators.python.PythonOperator
      python_callable_file: /scripts/transform.py
      python_callable_name: transform_sales
      op_kwargs:
        region: north
      dependencies: [extract_north_sales]
    load_north:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/load.py --region north'
      dependencies: [transform_north]

# ■■ DAG 2: South Region ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sales_pipeline_south:
  <<: *default_config
  description: 'Daily sales ETL for South Region'
  tags: [sales, south, daily]
  tasks:
```

```
    extract_south_sales:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/extract.py --region south --date {{ ds }}'
      dependencies: []
    transform_south:
      operator: airflow.operators.python.PythonOperator
      python_callable_file: /scripts/transform.py
      python_callable_name: transform_sales
      op_kwargs:
        region: south
      dependencies: [extract_south_sales]
    load_south:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/load.py --region south'
      dependencies: [transform_south]
```

dags/config/multi_pipeline.yml (Part 2 of 2)

```
# ■■ DAG 3: West Region (different schedule override) ■
sales_pipeline_west:
  <<: *default_config
  description: 'Daily sales ETL for West Region'
  schedule_interval: '0 10 * * 1-5'
  tags: [sales, west, daily]
  tasks:
    extract_west_sales:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/extract.py --region west --date {{ ds }}'
      dependencies: []
    aggregate_west:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/aggregate.py --region west'
      dependencies: [extract_west_sales]
    transform_west:
      operator: airflow.operators.python.PythonOperator
      python_callable_file: /scripts/transform.py
      python_callable_name: transform_sales
      op_kwargs:
        region: west
      dependencies: [aggregate_west]
    load_west:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/load.py --region west'
      dependencies: [transform_west]
```

TIP

YAML Anchors (&default_config) and aliases (<<: *default_config) let you define shared config once and inherit it across all DAGs — eliminating all duplication.
Override any key (e.g. schedule_interval for West) simply by re-declaring it under that DAG.

Step B2 — Advanced YAML: HttpSensor + BranchPythonOperator

dags/config/sensor_branch_pipeline.yml

dags/config/sensor_branch_pipeline.yml

```
# Demonstrates: HttpSensor, BranchPythonOperator, none_failed trigger rule

api_data_pipeline:
```

```
  description: 'API fetch with sensor and conditional branching'
  schedule_interval: '@hourly'
  start_date: '2024-01-01'
  catchup: false
  tags: [api, sensor, branching]
  default_args:
    owner: platform-team
    retries: 1
    retry_delay_sec: 60

  tasks:
    check_api_available:
      operator: airflow.providers.http.sensors.http.HttpSensor
      http_conn_id: api_connection
      endpoint: '/health'
      poke_interval: 30
      timeout: 300
      mode: reschedule
      dependencies: []

    fetch_data:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'curl -s $API_URL/data > /tmp/api_data.json'
      dependencies: [check_api_available]

    check_data_quality:
      operator: airflow.operators.python.BranchPythonOperator
      python_callable_file: /scripts/quality_check.py
      python_callable_name: check_quality
      dependencies: [fetch_data]

    process_valid_data:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/process.py --mode valid'
      dependencies: [check_data_quality]

    handle_invalid_data:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/process.py --mode quarantine'
      dependencies: [check_data_quality]

    finalize:
      operator: airflow.operators.empty.EmptyOperator
      trigger_rule: none_failed_min_one_success
      dependencies: [process_valid_data, handle_invalid_data]
```

Step B3 — Run and Verify All Generated DAGs

Terminal — Verify All DAGs

```
# Restart to pick up new YAML configs
astro dev restart

# List all registered DAGs
astro dev run dags list

# Expected output:
# dag_id                    | schedule        | tags
# order_etl_pipeline        | 0 6 * * *       | etl, orders, production
# sales_pipeline_north      | 0 8 * * 1-5     | sales, north, daily
```

```
# sales_pipeline_south      | 0 8 * * 1-5     | sales, south, daily
# sales_pipeline_west       | 0 10 * * 1-5    | sales, west, daily
# api_data_pipeline         | @hourly         | api, sensor, branching

# Trigger all regional DAGs
for region in north south west; do
  astro dev run dags trigger sales_pipeline_$region
done

# Monitor run status
astro dev run dags list-runs --dag-id sales_pipeline_north --limit 5
```

05 | MODULE C: DEPLOY PIPELINES TO ASTRONOMER ASTRO

Hands-on: Push your YAML DAGs to a live deployment

Pre-Deployment Checklist

#	Item	How to Verify
1	requirements.txt	dag-factory>=0.19.0 is listed
2	YAML syntax valid	No tab characters; consistent 2-space indentation throughout
3	Loader file exists	dags/dag_factory_loader.py loads from correct config path
4	Operators installed	All operator classes referenced in YAML are in requirements.txt
5	Connections set	All conn_id values exist in Astro UI Admin > Connections
6	Variables set	Airflow Variables used via Jinja are set in Admin > Variables
7	Astro login done	astro login completed; astro whoami shows your email

Step C1 — Authenticate with Astronomer Cloud

Terminal — Astro Authentication

```
# Login to Astronomer Cloud (browser OAuth flow)
astro login astronomer.io

# For self-hosted Astronomer Software:
astro login <your-basedomain>

# Verify authentication
astro whoami
# Output: Logged in as: your.email@company.com

# List all deployments in your workspace
astro deployment list
# NAME                  ID                   RUNTIME  STATUS
# prod-airflow-cluster  clxyz123456789       9.3.0    HEALTHY
# dev-airflow-cluster   clabc987654321       9.3.0    HEALTHY
```

Step C2 — Deploy to Astronomer Cloud

Terminal — Deploy to Astro Cloud

```
# Deploy current project to your active deployment
# (Run this from the project root where .astro/ folder exists)
astro deploy

# Specify a deployment explicitly by ID:
astro deploy --deployment-id clxyz123456789

# Non-interactive (for scripts and CI/CD):
astro deploy --force

# What happens during deploy:
```

```
# 1. Astro CLI builds your Docker image locally
# 2. Installs requirements.txt (including dag-factory)
# 3. Pushes image to Astronomer container registry
# 4. Triggers rolling restart of scheduler and webserver
# 5. New DAGs appear in Astro Cloud UI within 2-5 minutes

# Monitor deployment health after deploy
astro deployment inspect --deployment-id clxyz123456789
```

Step C3 — CI/CD with GitHub Actions

`.github/workflows/deploy-to-astro.yml`

.github/workflows/deploy-to-astro.yml

```yaml
# .github/workflows/deploy-to-astro.yml
name: Deploy DAG Factory Pipelines to Astronomer

on:
  push:
    branches: [main]
    paths:
      - 'dags/**'
      - 'requirements.txt'
      - 'Dockerfile'

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Install Astro CLI
        run: curl -sSL install.astronomer.io | sudo bash -s

      - name: Validate YAML configs
        run: |
          pip install yamllint dag-factory
          yamllint dags/config/*.yml
          python -c "
          import dagfactory, glob
          for f in glob.glob('dags/config/*.yml'):
            dagfactory.DagFactory(f)
          print('All configs valid')"

      - name: Deploy to Astronomer
        env:
          ASTRO_API_TOKEN: ${{ secrets.ASTRO_API_TOKEN }}
        run: |
          astro deploy ${{ secrets.ASTRO_DEPLOYMENT_ID }} --force
```

Step C4 — Verify in Astronomer Astro UI

Step	Where to Look	What to Confirm
1	Astro UI > DAGs tab	All YAML-generated DAGs listed with correct schedules
2	DAG > Graph View	Task graph matches the dependencies defined in YAML

3	Admin > Connections	All conn_id values referenced in YAML configs exist
4	Admin > Variables	Jinja-referenced variables are set with correct values
5	Toggle DAG on + Trigger	Manual run completes successfully end-to-end
6	Task Instance > Logs	Operator executed with expected output in logs
7	Deployment Overview	Scheduler heartbeat green; CPU/memory within normal range

06 | ADVANCED PATTERNS

Programmatic generation, Airflow Variables, SLA, and callbacks

Pattern 1 — Programmatic YAML Generation

When you need one DAG per database table or data source, generate YAML configs programmatically with a Python script instead of writing each by hand.

`include/generate_configs.py`

include/generate_configs.py

```
# include/generate_configs.py
# Run this script to auto-generate YAML DAG configs from a table list

import yaml
from pathlib import Path

TABLES = ['orders', 'customers', 'products', 'inventory', 'shipments']
OUTPUT_DIR = Path('dags/config/tables')
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

for table in TABLES:
    config = {
        f'etl_{table}_pipeline': {
            'description': f'Nightly ETL for {table} table',
            'schedule_interval': '0 2 * * *',
            'start_date': '2024-01-01',
            'catchup': False,
            'tags': ['auto-generated', table, 'nightly'],
            'default_args': {'owner': 'platform', 'retries': 2},
            'tasks': {
                f'extract_{table}': {
                    'operator': 'airflow.operators.bash.BashOperator',
                    'bash_command': f'python /scripts/extract.py --table {table}',
                    'dependencies': [],
                },
                f'transform_{table}': {
                    'operator': 'airflow.operators.bash.BashOperator',
                    'bash_command': f'python /scripts/transform.py --table {table}',
                    'dependencies': [f'extract_{table}'],
                },
                f'load_{table}': {
                    'operator': 'airflow.operators.bash.BashOperator',
                    'bash_command': f'python /scripts/load.py --table {table}',
                    'dependencies': [f'transform_{table}'],
                },
            }
        }
    }
    out = OUTPUT_DIR / f'{table}_pipeline.yml'
    with open(out, 'w') as f:
        yaml.dump(config, f, default_flow_style=False)
    print(f'Generated: {out}')

print(f'Done: {len(TABLES)} configs written to {OUTPUT_DIR}')
```

Pattern 2 — Airflow Variables in YAML

Dynamic Variables Pattern

```
# Use Airflow Variables via Jinja in YAML DAG configs
# Set variables in: Admin > Variables in Airflow UI

dynamic_config_pipeline:
  description: 'Pipeline driven by Airflow Variables'
  schedule_interval: '{{ var.value.etl_schedule }}'
  start_date: '2024-01-01'
  catchup: false
  tags: [dynamic, configurable]
```

```
  default_args:
    owner: '{{ var.value.pipeline_owner }}'
    retries: 2

  tasks:
    extract_data:
      operator: airflow.operators.bash.BashOperator
      bash_command: >
        python /scripts/extract.py
        --source {{ var.value.data_source_url }}
        --date {{ ds }}
        --env {{ var.value.environment }}
      dependencies: []

    load_data:
      operator: airflow.operators.bash.BashOperator
      bash_command: >
        python /scripts/load.py
        --target {{ var.value.db_connection_string }}
      dependencies: [extract_data]

# Variables to create in Admin > Variables:
# etl_schedule           0 6 * * *
# pipeline_owner         data-engineering
# data_source_url        https://api.example.com
# environment            production
# db_connection_string   postgresql://host/dbname
```

Pattern 3 — SLA Monitoring and Callbacks

SLA and Callbacks Pattern

```
# Add SLA enforcement and failure/success callbacks via YAML

sla_monitored_pipeline:
  description: 'Production pipeline with SLA and alerting'
  schedule_interval: '0 6 * * *'
  start_date: '2024-01-01'
  catchup: false
  sla_miss_callback: include.callbacks.sla_miss_handler
  on_failure_callback: include.callbacks.dag_failure_handler
  on_success_callback: include.callbacks.dag_success_handler
  tags: [sla, monitored, production]
  default_args:
    owner: sre-team
    retries: 3
    sla:
      seconds: 3600

  tasks:
    critical_job:
      operator: airflow.operators.bash.BashOperator
      bash_command: 'python /scripts/critical_process.py'
      sla:
        seconds: 1800
      on_failure_callback: include.callbacks.task_failure_handler
      dependencies: []

    post_process:
      operator: airflow.operators.bash.BashOperator
```

```
      bash_command: 'python /scripts/post_process.py'
      dependencies: [critical_job]
```

07 | TROUBLESHOOTING AND REFERENCE

Common errors, YAML schema cheatsheet, and essential CLI commands

Common Errors and Fixes

Error Symptom	Root Cause	Fix
DAG not appearing in UI	YAML parse error or wrong path	astro dev logs scheduler | grep ERROR Check 2-space indent, no tabs
ModuleNotFoundError: dagfactory	Package not in Docker image	Add dag-factory>=0.19.0 to requirements.txt Run astro dev restart
operator: ... not found	Wrong class path in YAML	Use full dotted path: airflow.operators.bash.BashOperator
Task has no upstream: error	dependencies key missing	Every task must have dependencies: [] even if no upstream tasks
Jinja template error	Variable not set or wrong syntax	Set Variable in Admin > Variables first Use {{ var.value.key }} syntax
Circular dependency error	Task A depends on B, B on A	Review all YAML dependency chains Draw the graph to spot the cycle
Deploy: image build failed	requirements.txt conflict	Test locally: astro dev start Check for version pin conflicts

Complete YAML Schema Reference

YAML Schema Reference (Part 1 of 2)

```yaml
# COMPLETE YAML DAG SCHEMA — Part 1: DAG-level keys

dag_id_here:                            # Required: becomes the Airflow DAG ID
  description: 'Human readable string'  # Optional
  schedule_interval: '0 6 * * *'        # Cron | @daily | @hourly | None
  start_date: 'YYYY-MM-DD'              # Required
  end_date: 'YYYY-MM-DD'                # Optional
  catchup: false                        # Boolean (default: false)
  max_active_runs: 1                    # Int — max concurrent DAG runs
  concurrency: 16                       # Int — max concurrent tasks
  tags: [tag1, tag2]                    # List of string tags
  doc_md: 'Markdown documentation'       # Optional DAG documentation
  on_failure_callback: module.function  # Dotted path to Python callable
  on_success_callback: module.function
  sla_miss_callback: module.function

  default_args:
    owner: 'team-name'
    depends_on_past: false
    retries: 2
    retry_delay_sec: 300
    email_on_failure: true
    email_on_retry: false
    email: [alerts@company.com]
    sla:
      seconds: 3600
```

YAML Schema Reference (Part 2 of 2)

```yaml
# YAML DAG SCHEMA — Part 2: Task-level keys

  tasks:
    task_id_here:                       # Required: unique task ID in this DAG
      operator: full.dotted.ClassName   # Required: full Python class path

      # BashOperator params:
      bash_command: 'echo hello {{ ds }}'

      # PythonOperator params:
      python_callable_file: /path/to/file.py
      python_callable_name: function_name
      op_kwargs:
        key: value
      op_args: [arg1, arg2]

      # Sensor params (HttpSensor example):
      http_conn_id: my_http_conn
      endpoint: '/health'
      poke_interval: 30
      timeout: 300
      mode: reschedule

      # Task-level control settings:
      trigger_rule: all_success         # all_success|one_failed|none_failed|etc
      retries: 3
      sla:
        seconds: 900
      on_failure_callback: module.func
      on_retry_callback: module.func

      # Dependencies — REQUIRED even when empty:
      dependencies: [upstream_task_id]  # List of upstream task IDs
```

Essential Commands Reference

Command	Purpose
astro dev start	Start local Airflow environment (webserver + scheduler + DB)
astro dev stop	Stop all local containers cleanly
astro dev restart	Restart to pick up requirements or config changes
astro dev logs scheduler	Tail scheduler logs — primary place to find DAG import errors
astro dev run dags list	List all registered DAGs and their schedules
astro dev run dags trigger <dag_id>	Manually trigger a DAG run
astro dev run dags test <dag_id> <date>	Test a DAG without writing to metadata DB
astro deploy	Deploy project to active Astronomer Cloud deployment
astro deploy --deployment-id <id>	Deploy to a specific deployment by ID
astro deployment list	List all deployments in your workspace
astro deployment inspect	Check health, status, config of a deployment

yamllint dags/config/	Lint all YAML files for syntax errors before deploying

SUCCESS

LAB COMPLETE: You can now (A) convert Python DAGs to YAML, (B) generate multiple DAGs from a single YAML config using anchors, and (C) deploy to Astronomer Astro via CLI + CI/CD.

Key files: dags/dag_factory_loader.py | dags/config/*.yml | requirements.txt

Next steps: Astronomer Registry at registry.astronomer.io for pre-built DAG Factory templates.

End of DAG Factory Lab Guide | Enterprise AI Training Program